Day-17 | Grover's Algorithm

Quantum Search & Amplitude Amplification

1. Learning Objectives

- ✓ Explain how quantum search differs from classical search.
- ✓ Understand the concepts of the **Oracle** and **Diffusion Operator**.
- ✓ Learn how **Amplitude Amplification** increases success probability.
- ✓ Visualize the workflow of Grover's Algorithm.
- ✓ Understand the quadratic computational speedup $\mathcal{O}(\sqrt{N})$.

2. Quick Recap

- **Superposition:** Creating an equal probability across all possible states using H gates.
- **Interference:** Combining waves to build up (constructive) or cancel out (destructive) probabilities.

Transition: Today, we use these principles to search an unsorted database significantly faster than a classical computer ever could.

3. Why Do We Need Grover's Algorithm?

Motivation: Searching through unstructured data (like finding a specific combination lock without clues) requires classical computers to check almost every possibility. Grover's algorithm provides a **provable quadratic speedup** to this fundamental problem.

Real-World Applications

- **Cryptography:** Searching for symmetric encryption keys (e.g., weakening AES-128 to 64-bit security).
- **Database Search:** Finding specific entries in massive, unstructured knowledge repositories.
- **Optimization & AI:** Accelerating the search phase in decision trees and constraint satisfaction problems.

4. Revisiting the Basics: Equal Superposition

Initial State Preparation

$$\psi = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} x$$

Where:

- N is the total number of possible solutions.
- x represents each individual candidate state.
- *Intuition:* The algorithm does not test one answer at a time; it starts by preparing **all** possibilities simultaneously with equal probability.

5. Concept Expansion: Myths vs. Reality

Common Misconception	Reality / Correction
"Grover's checks every answer simultaneously."	**Correction:** It prepares all states in superposition, but it manipulates *probabilities*, not parallel tests.
"The Oracle tells us the answer."	**Correction:** The Oracle never reveals the solution; it merely applies a hidden "tag" (a phase flip).
"More iterations = better results."	**Correction:** Over-iterating causes the probability of the correct answer to decrease again. You must stop at exactly $\approx \frac{\pi}{4}\sqrt{N}$ iterations.

6. Core Mechanism (Intuition First): Amplitude Amplification

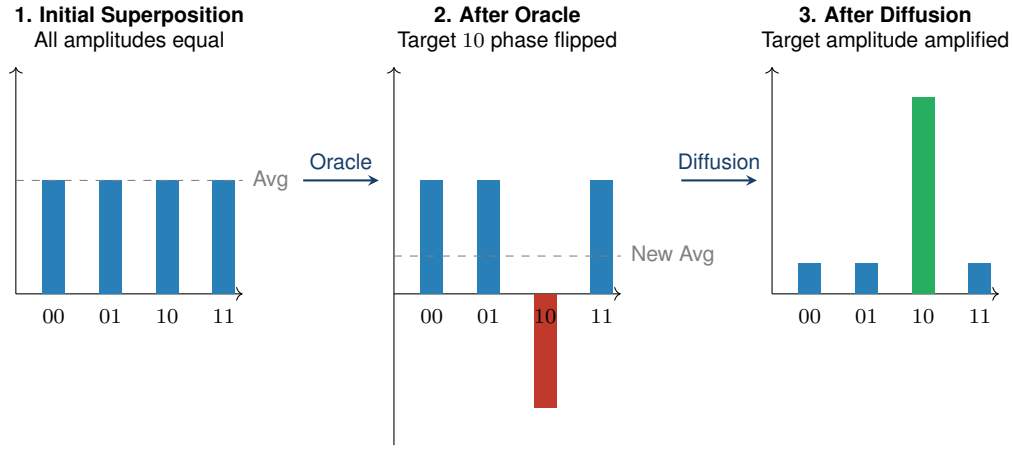
Grover's Algorithm is a repeated loop of two operations:

1. **The Oracle (The Tag):** Flips the *phase* (sign) of the correct answer to negative. The probability doesn't change, but it is now mathematically distinct.
2. **The Diffusion Operator (The Amplifier):** Reflects all amplitudes around the average. Because the correct answer is negative, the average is lowered. Reflecting around this new lower average makes the positive states smaller and the negative (correct) state much larger.

7. System Scaling & Complexity Advantage

Search Space (N)	Classical Checks $O(N)$	Grover Iterations $O(\sqrt{N})$
16 Items	≈ 8 to 16 checks	≈ 3 iterations
1,000,000 Items	$\approx 500,000$ to 10^6 checks	≈ 785 iterations
N Items	$O(N)$	$\approx \frac{\pi}{4}\sqrt{N}$

8. Visualizing the Process (4 Qubit Example)



9. Core Equations & Rules

1. Oracle Operator (O): $w \rightarrow -w$
Flips the sign of the target state w.
2. Diffusion Operator (D): $D = 2ss - I$
Reflects amplitudes about the average state s.
3. Grover Iteration (G): $G = D \cdot O$
Apply Oracle, then apply Diffusion.

10. Practical Implementation: Qiskit Circuit

We search for the specific state 11.

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler
from qiskit.circuit.library import GroverOperator

# 1. The Oracle (Tags state |11>)
oracle = QuantumCircuit(2)
oracle.cz(0, 1) # Controlled-Z flips phase only for |11>
oracle.name = "Oracle"

# 2. Build the Grover Operator (Combines Oracle + Diffusion)
grover = GroverOperator(oracle)

# 3. Main Circuit Setup
qc = QuantumCircuit(2, 2)
qc.h([0, 1])            # Step 1: Create equal superposition
qc.append(grover, [0,1]) # Step 2: Apply ONE Grover Iteration
qc.measure([0,1], [0,1]) # Step 3: Measure
```

```
# 4. Execution
sampler = StatevectorSampler()
job = sampler.run([qc], shots=1024)
counts = job.result()[0].data.meas.get_counts()
print(counts) # Output will heavily favor {'11': ~960}
```

11. ⋆ Key Takeaways

> Grover's Algorithm provides a **quadratic speedup** $\mathcal{O}(\sqrt{N})$ over classical unstructured search $\mathcal{O}(N)$.

> It does NOT test all items simultaneously. It manipulates **probability amplitudes**.

> The **Oracle** identifies the target by flipping its phase (negative sign).

> The **Diffusion Operator** performs "Amplitude Amplification," shrinking incorrect answers and magnifying the correct one.

> One complete cycle (Oracle + Diffusion) is a **Grover Iteration**.

> The number of iterations must be precisely calculated ($\approx \frac{\pi}{4}\sqrt{N}$); too many will decrease accuracy.